

```
import pandas as pd
import numpy as np

# To print numpy scalars as '3.0' rather than 'np.float64(3.0)'
np.set_printoptions(legacy="1.21")

def shortestPath(orig_node, df_nodes, df_links, forward_star):
    """
    Find the shortest paths from orig_node to all other nodes using Dijkstra's algorithm.

    Parameters
    -----
    orig_node      : int      """ the origin node ID
    df_nodes       : DataFrame """ node table (indexed by 'node')
    df_links       : DataFrame """ link table with 'cost' column (indexed by (init_node, term_node))
    forward_star   : dict     """ maps each node to its list of directly reachable downstream nodes

    Returns
    -----
    L              : dict """ shortest path cost from orig_node to each node
    q              : dict """ backnode of each node on the shortest path (-1 if not set)
    odpath         : dict """ key: (orig_node, dest_node), value: list of node IDs on the shortest path
    """

    # Initialise the label (cost) and backnode dictionaries
    L = {} # Label dictionary, shortest path cost from origin to each node
    q = {} # Backnode dictionary, preceding node on the current shortest path
    Q = [] # Queue to store all unvisited nodes

    # Loop through all nodes in df_nodes to initialise L, q, and Q
    for row in df_nodes.itertuples(index=False):
        L[row.node] = np.inf # Set the initial cost for all nodes to infinity
        q[row.node] = -1 # Initialise the backnode as -1 (no backnode initially)
        Q.append(row.node) # Add the node to the unvisited queue

    L[orig_node] = 0.0 # Set the shortest path cost of the origin node to 0

    # Main loop for Dijkstra's algorithm
    while len(Q) > 0: # Continue until all nodes have been visited
        i = -1 # Variable to hold the current node
        minL = np.inf # Initialize the minimum cost as infinity

        # Find the node with the smallest shortest path cost
        # in the unvisited queue Q
        for u in Q:
            if L[u] <= minL: # If the cost of u is less than the current minimum
                minL = L[u] # Update the minimum cost
                i = u # Set the current node to u

        Q.remove(i) # Remove the selected node i from the unvisited queue

        # Update L and q for all downstream nodes of the current node i
        # by looping through the forward star of node i
        for j in forward_star[i]:
            if j in Q: # If the downstream node j is still in the unvisited queue
                # Get the link travel time for traveling from node i to j
                t_ij = df_links.at[(i, j), "cost"]

                # Calculate the new shortest path cost to reach node j via node i
                newL = L[i] + t_ij
                if newL < L[j]: # If the new cost is smaller than the current cost
                    L[j] = newL # Update the shortest path cost for node j
                    q[j] = i # Set the backnode of node j to be node i

    # Create a odpath dictionary to store the shortest paths
    # between the origin and all other nodes (destinations)
    odpath = {} # key:(orig,dest), value:list of nodes in the shortest path

    # Reconstruct the shortest path for each destination node
    for row in df_nodes.itertuples(index=False):
        dest_node = row.node

        # Check if the destination node is reachable
        if dest_node == orig_node or L[dest_node] == np.inf:
            continue

        path = [dest_node] # Start with the destination node
        current = dest_node # Set the current node as the destination node

        # Trace back the path from the destination to the origin using the backnode dictionary q
        while q[current] != -1: # While there is a backnode to trace back
            # Insert the backnode of the current node at the start of the path
            path.insert(0, q[current])
            current = q[current] # Move to the previous node

        # Store the path in the odpaths dictionary
        odpath[(orig_node, dest_node)] = path

    return L, q, odpath
```